DEFENSE SCRIPT

Member 1 — Julius Baliling

Slides 1–13: Title → Framework Architecture

GA-LLaMEA Final Thesis Defense · April 2026

1

INTRODUCTION

Title Slide

0:30

Good morning, panel members. My name is Julius Baliling, and together with my groupmates Rigel Ray Cabaya and Keycee Rhaye Rivas, under the supervision of our adviser, Ms. Cheryll Pagal, we present our thesis entitled:

> "GA-LLaMEA: A Generative-Adaptive Large Language Model Evolutionary Algorithm — A Framework using Discounted Thompson Sampling for Adaptive Operator Selection in Automated Algorithm Design."

2

INTRODUCTION

Presentation Outline

0:30

To give you an overview of our presentation, we will cover seven sections. We begin with the Introduction and Problem Statement, followed by our Research Objectives. We then proceed to the Review of Related Literature, our Methodology, Results and Discussion, Ablation Studies, and finally our Conclusion and Recommendations.

I will be covering the Introduction, Literature Review, and Methodology sections — specifically slides one through thirteen. My groupmates will take over from the Results onward.

3

INTRODUCTION

Background of the Study

1:30

Before we discuss the specific problems, let us first establish the context of our study.

The fundamental challenge in optimization is designing metaheuristic algorithms — search strategies that solve complex mathematical problems. Traditionally, this has been done manually by domain experts, requiring extensive trial-and-error, deep mathematical intuition, and significant time investment. As Wolpert and Macready showed in their No Free Lunch theorem, no single algorithm dominates across all problem types — which means we constantly need new, specialized algorithms.

Large Language Models have changed this. LLMs can now generate executable optimization code directly from natural-language prompts, enabling a new paradigm called LLM-driven Automated Algorithm Design, or AAD. Frameworks like LLaMEA by van Stein and Bäck, and Evolution of Heuristics by Liu et al., treat the LLM as an evolutionary operator — the system iteratively prompts the model to refine and improve candidate algorithms, evaluates them on benchmarks, and selects the best performers.

However, despite these advances, current LLM-driven frameworks still have fundamental architectural limitations that restrict the quality and diversity of generated algorithms. These limitations are what motivated us to develop GA-LLaMEA — and they form the core of our problem statement.

The highlight box on screen summarizes the core insight — you can gesture toward it.

Our core insight is that by combining generative crossover — multi-parent recombination via LLM reasoning — with Discounted Thompson Sampling for adaptive operator selection, we can address these fundamental limitations.

4

INTRODUCTION

Problem Statement

1:30

Building on those limitations, we formalize three specific problems in LLM-driven Automated Algorithm Design.

First, underutilized populations. Current frameworks like LLaMEA rely on mutation-based refinement — each candidate is improved in isolation. There is no mechanism for recombining complementary strategies across the population, which leads to premature convergence.

Second, ineffective recombination. No general-purpose crossover operator exists within LLaMEA for continuous optimization. Existing syntactic crossover approaches — which simply concatenate code from two parents — produce logically inconsistent or structurally invalid offspring.

Third, the absence of adaptive operator selection. When you have multiple genetic operators, you need a way to decide which one to apply at each step. Static probabilities cannot respond to the shifting exploration-exploitation needs throughout an evolutionary run.

These problems are supported by the findings of Bouras et al., Hemberg and O'Reilly, and Aydin et al.

5

INTRODUCTION

Research Objectives

1:30

To address these problems, we defined three research objectives.

Objective one is to develop the GA-LLaMEA framework — a generative-adaptive evolutionary framework that integrates a modified bandit-based operator selection policy derived from Discounted Thompson Sampling, as proposed by Qi et al. in 2023.

Objective two is to implement Generative Crossover mechanisms — specifically, to upgrade the baseline LLaMEA architecture with a crossover operator that leverages LLM reasoning to synthesize coherent offspring from multiple parent strategies.

Objective three is to validate our framework using MA-BBOB benchmarking — evaluating performance through Elo ratings, AOCC or Area Over the Convergence Curve, behavioural profiling, Code Evolution Graphs, and operator selection dynamics.

For context, AOCC is scored in the range of zero to one, where higher values indicate faster and more consistent convergence across the evaluation budget.

6

INTRODUCTION

Significance of the Study

1:00

The significance of this study lies in three contributions.

First, it modernizes the LLaMEA framework by introducing Inspiration-Based Generative Crossover, which advances the framework's explorative capabilities in line with state-of-the-art population-based frameworks such as those by Meyerson et al. and Elsken et al.

Second, it pioneers the integration of bandit-based adaptive control into LLM-driven evolutionary optimization through Discounted Thompson Sampling, eliminating the need for fragile static operator schedules.

Third, it establishes rigorous empirical standards by validating generated algorithms using the MA-BBOB benchmark suite and standardized evaluation metrics including AOCC, as formalized by van Stein et al. in 2025.

7

INTRODUCTION

Scope & Limitations

1:00

In terms of scope — our study focuses on single-objective continuous black-box optimization using the BBOB suite of 24 noiseless functions. The generative component uses Gemini-2.5-flash via API with in-context learning only — no fine-tuning or weight retraining. All generated heuristics are written in Python and are COCO-compatible. The Discounted Thompson Sampling controller manages operator trade-offs using a discount factor gamma.

As for limitations — this study excludes combinatorial, large-scale global optimization problems beyond 100 dimensions, and multi-objective optimization. There are also computational budget constraints due to API costs and latency. Finally, while the framework is architecturally model-agnostic, it has been validated on Gemini-2.5-Flash only. Multi-model generalization remains a future direction.

8

LITERATURE

Section Divider: Review of Literature

0:15

We now move to Section Two — the Review of Related Literature, where we discuss the theoretical foundations and identify the specific research gaps that GA-LLaMEA addresses.

9

LITERATURE

Theoretical Foundations

2:00

Our theoretical foundation is built around four targeted questions.

First — what does current Automated Algorithm Design already do? Frameworks like LLaMEA and EoH generate metaheuristics via iterative LLM prompting with mu-plus-lambda selection. However, they remain mutation-centric — candidates evolve in isolation without cross-population information sharing. This is documented by van Stein and Bäck in 2024, and Liu et al. in 2024.

Second — why add an adaptive operator selector? With four operators in our framework, static probabilities cannot respond to shifting operator effectiveness. A bandit controller learns which operator is productive at each stage, eliminating fragile manual scheduling. This is supported by Aydin et al. and Fialho et al.

Third — why specifically D-TS over UCB or sliding-window approaches? UCB uses time-scheduled decay — exploration decreases based on elapsed evaluations. D-TS, on the other hand, uses exponential discounting with gamma equal to 0.99, responding to actual reward history rather than a clock. This makes it better suited for non-stationary environments, as shown by Qi et al. in 2023.

Fourth — why introduce crossover at all? Mutation alone restricts search to local basins, as Opris et al. demonstrate. While crossover exists in LLaMEA-BO and ReEvo, no prior work introduces a general-purpose crossover within LLaMEA for continuous black-box heuristic generation.

Pause briefly between each question — let the panel absorb each point.

10

LITERATURE

LLM-Driven Automated Algorithm Design

1:30

This table traces the evolution of Automated Algorithm Design — from Koza's foundational genetic programming in 1992, through hyper-heuristics in the 2000s, to modern LLM-driven frameworks.

Key milestones include FunSearch, published in Nature 2024, which combined LLMs with systematic evaluators; EoH by Liu et al. at ICML 2024, which co-evolves Python code and natural-language thought; and LLaMEA by van Stein and Bäck, which introduced LLM evolutionary algorithms for metaheuristic synthesis with feedback-driven refinement.

More recently, Zhang et al. in 2025 formalized the Algorithm Selection versus Generation taxonomy, positioning LLM-driven AAD at the research frontier. And CodeEvolve by Assumpção et al. in 2026 introduced multi-model code evolution with inspiration-based crossover.

Our work, GA-LLaMEA, is the first to combine general-purpose generative crossover with D-TS adaptive operator selection within the LLaMEA lineage.

You can see the research lineage at the bottom — from the original LLaMEA, through domain extensions like Photonic-LLaMEA, population-based extensions like LLaMEA-BO, structural feedback via LLaMEA-SAGE, and finally our contribution: GA-LLaMEA with generative crossover and D-TS.

Point to the highlighted GA-LLaMEA row, then the lineage timeline at the bottom.

11

LITERATURE

Research Gap — The Three Deficits

1:30

We identified three specific research gaps that our framework addresses.

Gap one — mutation-centric evolution. Current frameworks rely on single-parent refinement only. There is no mechanism to recombine complementary strategies across the population. Our solution is Inspiration-Based Generative Crossover, which enables multi-parent recombination at the strategy level.

Gap two — invalid recombination. Syntactic crossover, as used in existing approaches, produces structurally erroneous offspring because it operates without semantic constraints. Our solution is strategy-level semantic synthesis via LLM reasoning — the LLM reasons about algorithmic concepts, not raw code text.

Gap three — static operator scheduling. Fixed operator probabilities cannot adapt to non-stationary dynamics throughout an evolutionary run. Our solution is D-TS adaptive control with discounted posteriors, which learns in real-time which operator is most effective.

12

METHODOLOGY

Section Divider: Methodology

0:15

We now move to Section Three — the Methodology, where we present the framework architecture and experimental design.

13

METHODOLOGY

Framework Architecture

2:00

This slide shows a side-by-side comparison between the Baseline LLaMEA and our GA-LLaMEA framework.

On the left, we have the Baseline LLaMEA architecture by van Stein and Bäck. It follows a straightforward pipeline: initialization, population management, then random prompt selection — meaning the system randomly chooses between Refine, Random New, or Refine-Simplify operators. The selected operator prompts the LLM to generate code, which is then evaluated, and the population is updated. The key limitation here is that these are parallel but isolated refinement paths — there is no recombination across candidates, and operator selection is purely random.

Gesture toward the left card on screen.

On the right, we have our GA-LLaMEA framework. Two critical additions differentiate it from the baseline. **First**, we introduce Generative Crossover as a fourth operator — this enables multi-parent recombination, where the LLM synthesizes a new candidate by reasoning over strategy descriptions from multiple parents, not just refining a single one. **Second**, we replace random operator selection with Discounted Thompson Sampling — a bandit-based adaptive controller that learns from reward history which operator is most productive at each stage of the evolutionary run.

Gesture toward the right card on screen.

These two additions — generative crossover and D-TS — are the core architectural contributions of our framework.

— Hand off to next presenter —

That concludes the Introduction, Literature Review, and Framework Architecture sections. I will now hand over to my groupmate who will continue with the remaining methodology slides and results.

Timing Guide

SLIDE	TOPIC	EST. TIME
1	Title	0:30
2	Outline	0:30
3	Background of the Study	1:30
4	Problem Statement	1:30
5	Research Objectives	1:30
6	Significance	1:00
7	Scope & Limitations	1:00
8	Literature divider	0:15
9	Theoretical Foundations	2:00
10	LLM-Driven AAD	1:30
11	Research Gap	1:30
12	Methodology divider	0:15
13	Framework Architecture	2:00
	Total	**~15:00**

Delivery Tips

- On Slide 3 (Background), build the narrative: manual design → LLMs change everything → but limitations remain → our solution
- On Slide 9, pause briefly between each of the four questions — let each point land before moving on
- On Slide 10, point to the highlighted GA-LLaMEA row and the lineage timeline at the bottom
- On Slide 13, use hand gestures to contrast left (baseline) vs right (ours)
- Keep "D-TS" pronunciation consistent: spell it out as "D-T-S", not "dee-tis"
- AOCC: spell it out as "A-O-C-C"
- Speak slowly on the three research gaps — these map directly to your contributions